zkFuzz: Foundation and Framework for Effective Fuzzing of Zero-Knowledge Circuits (IEEE S&P'26)

Hideaki Takahashi, JIhwan Kim, Suman Jana, Junfeng Yang
Columbia University

Code : https://github.com/Koukyosyumei/zkFuzz
Project Site: https://zkfuzz.xyz/

Outline

What is Zero Knowledge Proof (ZKP)?

- ZKP allows us to prove that the statement is true

How do people implement ZKP in real-world products?

- We need to use DSLs to convert a statement into mathematical constraints.

Why does bugs arise in ZKP softwares?

- If the constraints are wrongly implemented, a malicious prover can prove wrong statements / a honest prover cannot prove true statements.

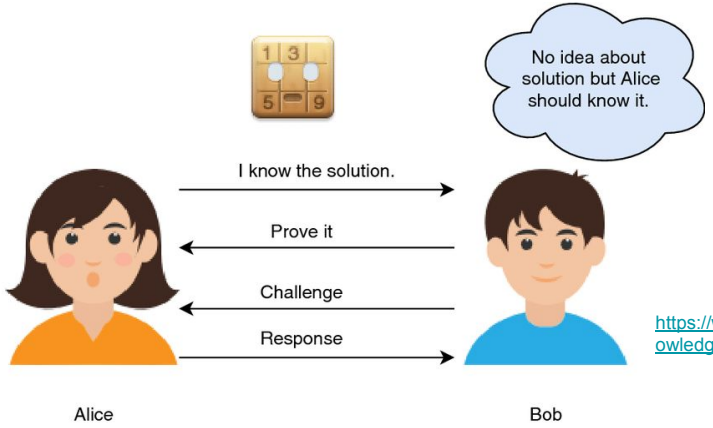
How can we identify bugs in ZKP softwares?

- zkFuzz, a fuzzer trying to detect discrepancies between programs and constraints.

What is Zero-Knowledge Proof (ZKP)?

Zero-Knowledge Proof (ZKP) is a concept in cryptography where one party proves to another that a statement is true, optionally without revealing any additional information beyond the fact that it is true.

- ***Prover***: the one who knows a secret (the witness)
- ***Verifier***: the one who wants to be convinced the statement is true



https://www.researchgate.net/figure/Interactive-zero-knowledge-proof_fig3_354455915

Properties of ZKP

Completeness

- If the statement is true and the prover knows a valid witness, an honest verifier will accept the proof with overwhelming probability.

Soundness

- If the statement is false, no cheating prover can convince the verifier to accept except with negligible probability.

Zero-knowledge (⚠️Optional)

- The verifier learns nothing from the interaction except that the statement is true

---

Succinctness

- Proof size is small, and verification is quick

Applications of ZKP

Scalable Blockchains (Validity Rollups) [1, 2]

- Transactions are verified off-chain using ZKP, and the resulting proof is succinctly verified on-chain.

Identification [3]

- Proving possession of a valid passport / ID card without revealing the passport itself.

zkML [4]

- Verifying the correctness of machine learning training or inference.

Software Distribution [5]

- Verifying that a binary was correctly compiled from the claimed code using the claimed compiler.

⚠️ *Note: Many zkXX projects (including this paper) do not actually rely on the zero-knowledge property.*

How Are ZKPs Implemented in Real-World Systems?

Modern ZKP protocols prove that:

- For a public constraint **C** and public output **y**, the prover knows an input **x** and some intermediate values **z** (witness) such that **C(x, z, y) = True**

In many ZKP protocols, constraints **C** must be expressed as polynomial constraints (ZK circuit) over the finite field **(mod p)**, often with degree limitations.

- Groth16 [6] works on **quadratic** polynomial constraints over the finite field.

Key Questions: How can complex statement / computations be represented as (quadratic) polynomial constraints?

Example: ZK Circuit for IsZero

Consider a simple program **IsZero**:

- It returns 1 if the input is 0
- It returns 0 otherwise

Can we design quadratic polynomial constraints **C** over a finite field such that

C(x, …, y) = True if and only if **IsZero**(x) = y?

Hint: You may introduce auxiliary (intermediate) variables.

Answer:

- **y - (-x * z + 1) = 0**
- **x * y = 0**

(mod 3)

x	z	y
0	0	1
0	1	1
0	2	1
1	1	0
2	2	0

Typical Development Flow of ZKP

In practice, developers implement two components simultaneously:

Witness Generation Program: P(x) = (z, y)

- A program that takes an input **x**, returning the output **y** while computing some intermediate values **z**.
- It may contain arbitrary computations like if-else, division, assert, e.t.c,.

ZK Circuit (Polynomial Constraints): **C(x, z, y) = true/false**

- A set of polynomial constraints such that **C(x, _, y) = True** iff **P(x) = (_, y)**
- In many proof systems, these constraints must be low-degree polynomials, typically <u>quadratic</u>.

Development of Constraints is Difficult

```
[ 21888242871839275222246405745257275088548364400416034343698204186575808495616 1 +main.gt.lt.n2b.out[0] ] * [ main.gt.lt.n2b.out[0] ] - [  ] = 0
[ 21888242871839275222246405745257275088548364400416034343698204186575808495616 1 +main.gt.lt.n2b.out[1] ] * [ main.gt.lt.n2b.out[1] ] - [  ] = 0
[ 21888242871839275222246405745257275088548364400416034343698204186575808495616 1 +main.gt.lt.n2b.out[2] ] * [ main.gt.lt.n2b.out[2] ] - [  ] = 0
[ 21888242871839275222246405745257275088548364400416034343698204186575808495616 1 +main.gt.lt.n2b.out[3] ] * [ main.gt.lt.n2b.out[3] ] - [  ] = 0
[ 21888242871839275222246405745257275088548364400416034343698204186575808495616 1 +main.gt.lt.n2b.out[4] ] * [ main.gt.lt.n2b.out[4] ] - [  ] = 0
[ 21888242871839275222246405745257275088548364400416034343698204186575808495616 1 +main.gt.lt.n2b.out[5] ] * [ main.gt.lt.n2b.out[5] ] - [  ] = 0
[ 21888242871839275222246405745257275088548364400416034343698204186575808495616 1 +main.gt.lt.n2b.out[6] ] * [ main.gt.lt.n2b.out[6] ] - [  ] = 0
[ 18297203025678144131096604802676003394333398365972778709185217562215714914304 1 +2main.oldEnough +17100189743624433767380004488482246162928409687825026831014222020762350387 2main.gt.lt.n2b.out[0] +34200379487248867534760
20207623503872main.age +17100189743624433767380004488482246162928409687825026831014222020762350387 2main.gt.lt.n2b.out[0] +34200379487248867534760
5681937565005366202844404152470077 44main.gt.lt.n2b.out[1] +68400758974497735069520017953928984651713638751300107324056888083049401548 8main.gt.lt.
0151794899547013904003590785796930342727750260021464811377616609880309 76main.gt.lt.n2b.out[3] +27360303589799094027808007181571593860685455500520
1976061952main.gt.lt.n2b.out[4] +54720607179598188055616014363143187721370911001040085859245510466439521239 04main.gt.lt.n2b.out[5] +10944121435 91
6286375442741822002080171718491020932879042478 08main.gt.lt.n2b.out[6] ] * [ 18297203025678144131096604802676003394333398365972778709185217562215 7
oldEnough +17100189743624433767380004488482246162928409687825026831014222020762350387 2main.age +17100189743624433767380004488482246162928409687 8
07623503872main.gt.lt.n2b.out[0] +34200379487248867534760008976964492325856819375650053662028444041524700774 4main.gt.lt.n2b.out[1] +68400758974 49
928984651713638751300107324056888083049401548 8main.gt.lt.n2b.out[2] +13680151794899547013904003590785796930342727750260021464811377616609880309 76
ut[3] +27360303589799094027808007181571593860685455500520042929622755233219760619 52main.gt.lt.n2b.out[4] +54720607179598188055616014363143187721 3
9245510466439521239 04main.gt.lt.n2b.out[5] +10944121435919637611123202872628637544274182200208017171849102093287904247808main.gt.lt.n2b.out[6] ]

[ 21888242871839275222246405745257275088548364400416034343698204186575808495616main.oldEnough ] * [ 1 +21888242871839275222246405745257275088548
98204186575808495616main.oldEnough ] - [  ] = 0
```

Polynomial constraints for `**age > 21**`

⇒ We use DSLs to make it easier to implement such constraints.

Domain-Specific Language (DSL) for ZKP

Real-world ZK projects usually use **DSL** to implement a witness generation program and a circuit (polynomial constraints).

- Circom, Zokrates, Noir, Aleo, Plonky3, …

```
template Multiplier2 () {

   // Declaration of signals.
   signal input a;
   signal input b;
   signal output c;

   // Constraints.
   c <== a * b;
}
```

Witness generation program

```
def (a, b):
  c = a * b
  assert(c, a * b)
  return c
```

Polynomial constraints

$c - a * b = 0$

Example Workflow of Circom

Step 1: Compile *.cirocm file

Witness generator (web assembly)

```
circom multiplier2.circom --r1cs --wasm --sym --c
```

constraint

Step 2: Calculate witness (variable assignments)

input witness

```
node generate_witness.js multiplier2.wasm input.json witness.wtns
```

Step 3: Generate a final proof

```
snarkjs groth16 prove multiplier2_0001.zkey witness.wtns proof.json public.json
```

Why Are ZK DSLs Difficult to Use?

ZK DSLs, such as Circom, often have non-intuitive semantics.

`x === y`: (equality constraint)

- Add the constraint **x - y = 0**
- Insert **assert(x, y)** into the witness generation program

If false, the witness generation program P abnormally terminates.

`x <== y`: (strong assignment)

- Add the constraint l **x - y = 0**
- Insert the assignment **let x = y** and **assert(x, y)** into the witness generation program

`x <-- y`: (weak assignment)

- Insert the assignment **let x = y** into the witness generation program

`assert(x, y)`:

- Insert the **assert(x, y)** into the witness generation program

Example: IsZero in Circom

```
1 template IsZero() {
2  signal input x;
3  signal output y;
4  signal z;
5  z <-- x != 0 ? 1/x : 0;
6  y <== -x*z + 1;
7  x*y === 0;
8 }
```

Witnee generation program

```
def (x, y):
  if x != 0
    z = 1 / x
  else
    z = 0
  y = -x * z + 1
  assert(y, -x * z + 1)
  assert(x * y, 0)

  return y
```

Polynomial constraints

- **y - (-x * z + 1) = 0**
- **x * y = 0**

Example of Circom Bug - ①

https://github.com/tokamak-network/DEPRECATED-zvm

```
1 template Transfer(){
2  signal input fb, tb;
3  signal input amt;
4  signal output fn, tn
5  assert(fb - amt >= 0);
6  fn <== fb - amt;
7  tn <== tb + amt;
8 }
```

Intention:

- Given the sender's balance (**fb**), the receiver's balance (**tb**), and the transfer amount (**amt**), compute the post-transfer balances of the sender and receiver (**fn**, **tn**), while asserting **fb > amt**.

Problem:

- The **assert** instruction only inserts an assertion into the witness generation program and <u>does **not** create any polynomial constraints.</u>

⇒ Therefore, a proof can still be generated even when **fb < amt**.

Example of Circom Bug - ②

https://github.com/rarimo/passport-zk-circuits/pull/60

```
template DateEncoder() {
 signal output encoded;
 signal input  day, month, year;
 signal dayD <-- (day \ 10);
 signal dayR <-- (day % 10);
 dayD * 10 + dayR === day;
 signal dayE <== (dayD * 2**8 + dayR) + 12336;
//Encode month and year in the same manner
 encoded<== yearE * 2**32 + monthE * 2**16 + dayE;
```

It decompose **day** into

- tens digit **dayR**
- ones digit **dayD**

such that

day = 10·dayR + dayD

Consider mod 11.

- Suppose **day = 4**
- Valid: {**dayD:0**, **dayR:4**}
- Attack: {**dayD: 1**, **dayR: 5**}

(1 * 10 + 5 = 15 = 4 (mod 11))

Without additional range constraints, multiple invalid witnesses can satisfy the circuit, which means that it is under-constrained.

Types of Bugs in ZK Circuits

Bugs in ZK circuits are typically categorized into two types.

Under-Constrained Bug: Too weak, allowing invalid witnesses to satisfy them.

- **P(x) = (_, y)**, while **C(x, _, y') = true** for some **y'** such that **y' \neq y**
- **P(x)** is expected to fail, while **C(x, _, y) = true** for some **y**

Over-Constrained Bug: Too strong, rejecting valid witnesses.

- **P(x) = (z, y)**, while **C(x, z, y) = false**

We are the first work proposing a unified formulation to capture all of them

Our Formulation - ZK Program

Definition 1 (ZK Program). *A ZK program is defined as a pair of a computation and a set of circuit constraints* $(\mathcal{P}, \mathcal{C})$*:*

$\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^k \times (\mathbb{F}_q^m \cup \{\perp\})$ *is a computation that takes as input a tuple* $(x_1, x_2, \ldots, x_n) \in \mathbb{F}_q^n$*, and behaves as follows:*

- *On successful termination, outputs a pair* (z, y) *where* $z = (z_1, \ldots, z_k) \in \mathbb{F}_q^k$ *represents intermediate values necessary to construct a witness and* $y = (y_1, \ldots, y_m) \in \mathbb{F}_q^m$ *the final outputs. Each value is assigned exactly once.*
- *On abnormal termination, it returns the special value* $(_, \perp)$.

$\mathcal{C} : \mathbb{F}_q^n \times \mathbb{F}_q^k \times \mathbb{F}_q^m \to \{true, false\}$ *is a set of circuit constraints, which evaluates to* `true` *if the given input, intermediate, and output values satisfy the required conditions; and* `false` *otherwise.*

P(x) = (z, y) or ⊥

Our Formulation - Trace Set and Constraint Satisfaction Set

Definition 3 (Trace Set). *For a computation $\mathcal{P}$, the trace set $\mathcal{T}(\mathcal{P})$ is defined as:*

$$\mathcal{T}(\mathcal{P}) := \{(x, z, y) \mid x \in \mathbb{F}_q^n, z \in \mathbb{F}_q^k, y \in \mathbb{F}_q^m, \mathcal{P}(x) = (z, y)\}$$

T(P) is the set of all traces that P can generate.

Definition 4 (Constraint Satisfaction Set). *For circuit constraints $\mathcal{C}$, the constraint satisfaction set $\mathcal{S}(\mathcal{C})$ is defined as:*

$$\mathcal{S}(\mathcal{C}) := \left\{ (x, z, y) \;\middle|\; \begin{array}{l} x \in \mathbb{F}_q^n,\ z \in \mathbb{F}_q^k,\ y \in \mathbb{F}_q^m, \\ C(x, z, y) = \mathit{true} \end{array} \right\}$$

S(C) is the set of all assignments that C can accept.

Our Formulation - Projection

Definition 5 (Projection). *Let* $\{(x_1, z_1, y_1), \ldots\} \subseteq 2^{\mathbb{F}_q^n \times \mathbb{F}_q^k \times \mathbb{F}_q^m}$ *be a set of tuples of inputs, intermediates and outputs. Then, we define the projection operator* Π_{xy} *as* $\Pi_{xy}(\{(x_1, z_1, y_1), \ldots\}) := \{(x_1, y_1), \ldots\}$.

This projection simply ignores the intermediate values.

Our Formulation - Trace-Constraint Consistency Test

Definition 6 (Under-Constrained Circuit). *We say that $\mathcal{C}$ is under-constrained for $\mathcal{P}$ if*

$$\Pi_{xy}(\mathcal{S}(\mathcal{C})) \setminus \Pi_{xy}(\mathcal{T}(\mathcal{P})) \neq \emptyset \tag{1}$$

Invalid input-output pairs satisfy the constraints.

Definition 7 (Over-Constrained Circuit). *We say that $\mathcal{C}$ is over-constrained for $\mathcal{P}$ if*

$$\mathcal{T}(\mathcal{P}) \setminus \mathcal{S}(\mathcal{C}) \neq \emptyset \tag{2}$$

Valid witness cannot satisfy the constraints.

Comparison of Formal Definitions

TABLE 1: Comparison of formal definitions of ZK vulnerabilities. Our TCCT is more general than existing models.

Model	Under-Constrained	Over-Constrained
[18], [25]	$\exists x, z, z', y, y'.$ $y \neq y' \wedge C(x, z, y) \wedge C(x, z', y')$	-
[24]	Same as [18], [25]	$\mathcal{S}(\mathcal{C}) = \emptyset$
TCCT	$\Pi_{xy}(\mathcal{S}(\mathcal{C})) \setminus \Pi_{xy}(\mathcal{T}(\mathcal{P})) \neq \emptyset$	$\mathcal{T}(\mathcal{P}) \setminus \mathcal{S}(\mathcal{C}) \neq \emptyset$

Existing models for under-constrained circuit only focus whether there are **multiple input-output** pairs that satisfy constraints, and ignores the case where **P(x)** is expected to fail, while **C(x, _, y) = true** for one **single y**.

Our formulation, TCCT, has been adopted in a recent work by another team [7].

Research on Detecting Bugs in ZK Circuits

Pattern-Matching-Based Static Analysis

- Circomspect, ZKAP (USENIX’23)
- Lightweight and easy to use, while producing many false positives.

SMT-Solver-Based Formal Verification

- ConsCS (ICSE’25), Picus (PLDI’23)
- Can detect bugs and also prove correctness, while tending to be too slow.

Interactive Theorem Proving (Coq, Lean4)

- CODA (S&P’24), CIVER (S&P’24)
- Can formally prove correctness, while requiring human interaction

Dynamic Testing / Fuzzing-Based Bug Detection

- zkFuzz (S&P’26)
- Can find bugs relatively quickly without producing any false positives.

Our Approach: zkFuzz

Algorithm 1 Core Logic of ZKFUZZ

```
for i ← 1, 2, ··· MAX_GENERATION do
    Generate input data x.
    Mutate P to P'
    Execute both P and P' on x:
        P(x) = (z, y), P'(x) = (z', y').
    if y ≠ ⊥ and C(x, z, y) = false then
        Report "Over-Constrained Problem."
    if y' ≠ ⊥ and y ≠ y' and C(x, z', y') = true then
        Report "Under-Constrained Problem."
```

Given a witness generation program **P** and a constraint **C**, **we try to find their discrepancies.**

Mutate both inputs and the target program a little bit to yield invalid witness, checking whether the constraint accept it or not.

Our Approach: Target Selector

To focus on promising candidates, **zkFuzz** adopts several strategies:

Skewed Input Generation

- Draw random inputs from skewed distribution, focusing on the edge-cases near zero or the field characteristic (prime number **p**).

Weak Assignment (`<--`)

- Focus on mutating the right-hand side of weak assignment (`<--`), which does not influence the constraint, and safely modifying only the program.

White List

- Skip mutating some commonly used templates whose correctness has been already formally guaranteed (`IsZero`, `Num2Bits`).

Evaluation

Benchmark: 452 realworld test suites written in Circom

Result: Detecting 85 of 88 vulnerabilities, including 59 previously unknown bugs.

Bug Bounties: $x000 from several teams/companies

Detection Speed: Identifies 90% of bugs within just 100 seconds.

Third-Party

program abort bugs. On our benchmark, ZKFUZZ identified 93 bugs (29 non-deterministic, 64 computation abort), averaging 152.44 seconds per bug circuit. In comparison, SCALECIRC identified 24 non-deterministic bugs, averaging 10.59 seconds per bug circuit. ZKFUZZ found 7 non-deterministic bugs that SCALECIRC missed, while SCALECIRC found 2 that ZKFUZZ missed. Overall, ZKFUZZ identifies more bugs, but

Evaluation

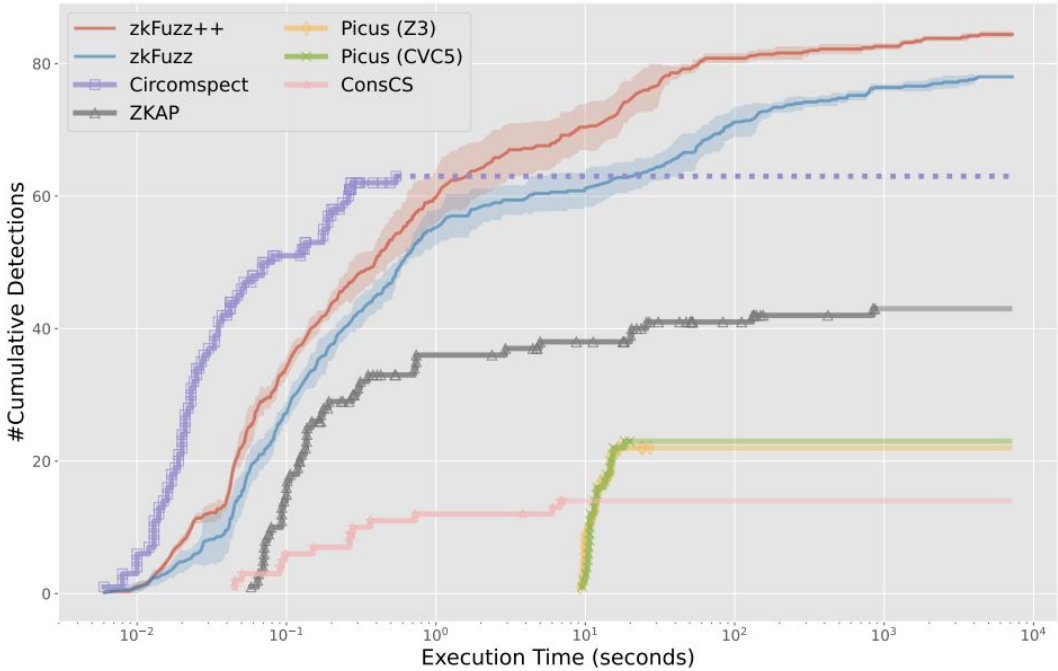


Figure 3: Detection Time Analysis. Both modes of ZKFUZZ are superior to other methods.

zkFuzz++

- zkFuzz + target selectors

SMT-based verifier

- Picus, ConsCS:

Static Analyzers

- Circomspect, ZKAP

Conclusion

Code : https://github.com/Koukyosyumei/zkFuzz
Project Site: https://zkfuzz.xyz/

Our Contributions

- **TCCT**: a language-independent bug model for ZK Circuit
- **zkFuzz**: a fuzzing framework for finding ZK circuit bugs by guided mutations and inputs to detect under- and over-constrained circuits without false positives.

Key Takeaways

- **Implementing ZK circuits** (polynomial constraints) that correctly match the intended computation is **highly error-prone**.
- **Discrepancies between the witness generation program and the constraints** provide a unified perspective for modeling ZK circuit bugs.
- **Fuzzing is an effective** technique for quickly uncovering bugs in ZK circuits.

Reference

[1] Donno, Luca. "Optimistic and validity rollups: Analysis and comparison between optimism and starknet." arXiv preprint arXiv:2210.16610 (2022).

[2] Katsika, Angeliki, et al. "A critical view on blockchain rollups." Security and Privacy in Smart Environments. Cham: Springer Nature Switzerland, 2024. 204-239.

[3] zkPassport, https://github.com/zkpassport

[4] Peng, Zhizhi, et al. "A survey of zero-knowledge proof based verifiable machine learning." arXiv preprint arXiv:2502.18535 (2025).

[5] Ron, Javier, and Martin Monperrus. "Verifiable Provenance of Software Artifacts with Zero-Knowledge Compilation." arXiv preprint arXiv:2602.11887 (2026).

[6] Groth, Jens. "On the size of pairing-based non-interactive arguments." Annual international conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016.

[7] Fu, Rong, et al. "zkCraft: Prompt-Guided LLM as a Zero-Shot Mutation Pattern Oracle for TCCT-Powered ZK Fuzzing." arXiv preprint arXiv:2602.00667 (2026).